

SOFTWARE REQUIREMENTS SPECIFICATION

Full-Stack Real-Time Chat Application

Two-Developer Implementation Plan - Frontend + Backend
5-Day Integrated MVP Delivery

Document Item	Value
Design basis	User-supplied Figma screenshots showing Profile, Chats, Calls, Groups, Shared Media, Starred Messages, Settings and dashboard states
Frontend stack	Next.js (React) + TypeScript + Tailwind CSS + Redux Toolkit + Socket.IO Client + WebRTC
Backend stack	Node.js + Express.js + TypeScript + MongoDB/Mongoose + Socket.IO + JWT
Media	Cloudinary or equivalent object storage
Target delivery	5 calendar days; integrated MVP, with advanced production hardening treated separately
Team	Developer 1 - Frontend Developer 2 - Backend

Delivery reality

The 5-day target is achievable only as a tightly scoped integrated MVP with focused work from both developers, fast decisions, prepared design assets, and no major infrastructure blockers. Full production-grade calling, security hardening, moderation, scaling and exhaustive device testing may extend beyond the five-day window.

Version: 1.0

Prepared for: Two-person development team

Document purpose: Convert the supplied design into an implementable, coordinated full-stack plan with exact sequencing, contracts, integration points and a five-day schedule.

Document Contents

1. Executive Summary and Objectives
2. Product Scope and Design Interpretation
3. User Roles and Primary User Flows
4. System Architecture
5. Frontend Requirements and Work Breakdown
6. Backend Requirements and Work Breakdown
7. API Contract and Data Contract
8. Database / Data Model Specification
9. Real-Time Messaging Specification
10. Calling and WebRTC Specification
11. Media / File Handling Specification
12. Settings, Privacy and Security Requirements
13. Frontend-Backend Coordination and Integration Method
14. Git / Branching / Pull Request Workflow
15. Five-Day Detailed Execution Plan
16. Daily Integration Checkpoints
17. Testing and Acceptance Criteria
18. Definition of Done
19. Risks, Constraints and Stretch Scope
20. Appendices - Checklists, Naming and Handoff Templates

Core rule for the team

Do not wait for one side to finish the whole application. Work feature-by-feature: agree the contract -> build frontend and backend in parallel -> integrate -> test -> move to the next feature.

1. Executive Summary and Objectives

This SRS defines a full-stack real-time messaging application based on the supplied UI/UX screens. The visible design includes a desktop-style communication workspace with chat lists, conversation area, profile management, group creation, audio/video calls, call logs, updates, shared media, starred messages, and multiple privacy/security/settings views.

The system will be implemented as a web application using Next.js/React on the frontend and a Node/Express TypeScript API on the backend. MongoDB will persist application data. Socket.IO will provide real-time events such as messaging, typing, presence and call signaling. WebRTC will carry live audio/video media between participants after signaling is completed.

1.1 Objectives

- Provide a visually faithful implementation of the supplied design screens.
- Enable authenticated one-to-one messaging in real time.
- Support groups, group membership, media sharing, starred messages and call history.
- Provide audio and video calling through WebRTC with Socket.IO signaling.
- Provide profile, theme, privacy, security and blocked-contact settings.
- Use Redux Toolkit for predictable client state and a clear API/service layer for backend communication.
- Finish an integrated MVP in five days through strict parallel development and daily integration.

1.2 Success Criteria

Criterion	Definition of success
Visual	Key Figma screens are reproduced closely on desktop and behave correctly at agreed responsive breakpoints.
Functional	User can register/login, open chats, send/receive messages, create groups, upload media and manage settings.
Real-time	Messages and presence/typing events update without manual refresh.
Calls	A tested peer-to-peer audio/video call flow works in supported modern browsers, subject to WebRTC network conditions.
Coordination	Frontend and backend use a written API/socket contract; integration defects are tracked and resolved daily.
Delivery	A clean develop branch contains the integrated MVP and main contains the tested release candidate.

2. Product Scope and Design Interpretation

The requirements below are derived from the supplied screenshots and expanded only where a full-stack implementation requires an API, data model, state model or technical behavior that is not directly visible in the UI.

2.1 Visible Modules in the Supplied Design

Area	Visible design states/screens	Primary backend capability
Profile	Profile / edit profile style screen	User profile CRUD, avatar, about, account preferences
Chats	Initial screen, chat list, conversation workspace	Chat membership, messages, unread counts, search support
New Conversation	Start new conversation modal/state	User/contact lookup, create or open 1:1 chat
Calls	Audio call, video call, call logs	Call session records + WebRTC signaling
Groups	Group list, create group, group details	Group CRUD, membership and permissions
Dashboard / Updates	Updates, filter chats, view update,	Status/update records, visibility and

Area	Visible design states/screens	Primary backend capability
	create/edit update	interactions if approved
Contact Info	Contact information screen	User read model, shared media, blocked state
Shared Content / Media	Media/files content views	Attachment metadata, object-storage URLs
Starred Messages	Starred message view	Message star/unstar and query
Settings	Settings, Theme, Keyboard Shortcuts, Privacy, Last Seen, Profile Photo, About, Groups, Blocked Contacts, Security	Preference persistence, privacy policy, blocks, sessions/security controls

2.2 MVP Scope for Five Days

Must-have for Day 5	Can be simplified for MVP	Stretch / post-MVP
Register/login/logout; 1:1 chat; real-time send/receive; typing; read state; groups; profile; shared media; starred messages; basic call log; basic audio/video calling; core settings; responsive layout	Basic search, simplified privacy controls, simple admin/member groups, limited call state reporting, and reduced non-critical animations.	Advanced E2E encryption; multi-device sync; advanced moderation; forwarding/edit history; advanced media optimization; push notifications; call recording; screen sharing; large-scale pagination tuning; enterprise audit/compliance

3. User Roles and Primary User Flows

3.1 Roles

Role	Capabilities
Authenticated User	Manage own profile/settings, search contacts, create chats, send messages, create/join groups, share media, star messages, make/receive calls, block contacts.
Group Admin	All user permissions plus add/remove members, update group information and manage group membership.
System / API	Validates authentication, permissions, storage operations, real-time events, and data integrity. No separate admin UI is required for the five-day MVP unless the team explicitly adds it.

3.2 Primary Flow - Login to Chat

1. User opens the Next.js application.
2. Frontend checks auth state; if no valid session exists, Login page is displayed.
3. User submits credentials to POST /api/auth/login.
4. Backend validates credentials and returns authenticated user data plus token/session data.
5. Frontend stores the authenticated state in Redux and routes to the main workspace.
6. Frontend requests chats from GET /api/chats and establishes a Socket.IO connection.
7. User opens a conversation; frontend loads recent messages and subscribes to chat events.
8. User sends a message; frontend emits the agreed Socket.IO event and/or calls the message API according to the final chosen pattern.
9. Backend persists the message, emits message:created to authorized participants, and returns a server-generated message ID/time.
10. Recipient UI updates immediately without refresh.

3.3 Primary Flow - Start New Conversation

1. User clicks New Conversation.

2. Frontend queries users/contacts by search string.
3. User selects another user.
4. Frontend calls POST /api/chats or an idempotent open-or-create endpoint.
5. Backend returns the existing or newly created 1:1 chat ID.
6. Frontend navigates to the conversation view and loads messages.

3.4 Primary Flow - Create Group

1. User opens Groups/Create Group.
2. Frontend collects group name, avatar (optional) and initial member list.
3. Frontend sends POST /api/groups.
4. Backend creates group, membership records and returns group data.
5. Frontend updates Redux group/chat state and opens the group conversation.

3.5 Primary Flow - Call

1. Caller opens a chat and starts audio/video call.
2. Frontend requests camera/microphone permissions.
3. Frontend creates a WebRTC PeerConnection.
4. Caller sends call invite plus SDP offer through Socket.IO signaling.
5. Receiver accepts/rejects; on accept, receiver creates answer and exchanges ICE candidates.
6. Once ICE/SDP negotiation succeeds, browser media flows peer-to-peer where network conditions permit.
7. Backend records call state/timestamps and call outcome for call logs.

4. System Architecture

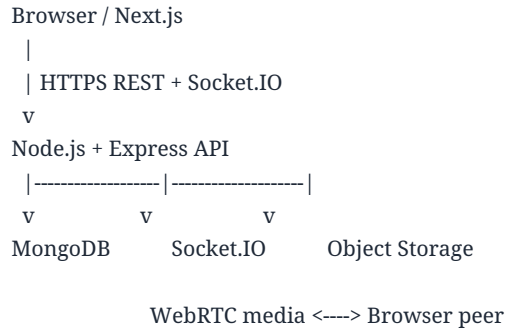
Recommended architecture

Use one repository with two applications/packages: /frontend and /backend, plus /docs for the API contract. This minimizes coordination overhead during the five-day delivery.

```
chat-app/
  frontend/
    app/
      components/
      features/
      store/
      services/
      types/
  backend/
    src/
      config/
      controllers/
      middleware/
      models/
      routes/
      services/
      sockets/
      utils/
      types/
  docs/
    API_CONTRACT.md
    SOCKET_CONTRACT.md
```

DATA_MODEL.md
README.md

4.1 Component Diagram



4.2 Environment Variables

Application	Variable	Purpose
Frontend	NEXT_PUBLIC_API_URL	Base URL for REST API
Frontend	NEXT_PUBLIC_SOCKET_URL	Socket.IO server URL
Backend	PORT	API port, e.g. 5000
Backend	MONGODB_URI	MongoDB connection string
Backend	JWT_SECRET	Token signing secret
Backend	CLOUDINARY_CLOUD_NAME / API_KEY / API_SECRET	Media storage configuration if Cloudinary is used
Backend	CLIENT_URL	Allowed frontend origin for CORS
Optional	TURN_SERVER_*	WebRTC relay configuration if direct peer-to-peer connectivity fails in some networks

Secret handling

Commit a .env.example with placeholder names only. Keep real secrets in local .env/.env.local and deployment Environment Variables; never commit API keys, JWT secrets, database credentials or storage secrets.

5. Frontend Requirements and Work Breakdown

Developer 1 owns the Next.js application, UI fidelity, responsive behavior, Redux state, API integration, Socket.IO client behavior, and WebRTC client behavior. Backend work should not block UI construction; use typed mock data until the corresponding endpoint is ready.

5.1 Frontend Folder Structure

frontend/src or app-based equivalent

- app/
 - (auth)/login/
 - (auth)/register/
- chat/
- groups/
- calls/
- settings/
- profile/
- components/
- layout/
- chat/

groups/
 calls/
 media/
 settings/
 ui/
 features/
 auth/
 chats/
 messages/
 groups/
 calls/
 notifications/
 settings/
 store/
 store.ts
 slices/
 services/
 api.ts
 authApi.ts
 chatApi.ts
 groupApi.ts
 uploadApi.ts
 socket.ts
 types/
 api.ts
 socket.ts
 models.ts

5.2 Frontend Module Requirements

Module	Requirements
Authentication	Login, register, logout, protected routes, auth bootstrap, validation, loading/error states.
Main Workspace	Desktop three-panel style layout where appropriate; sidebar/navigation, chat list, active conversation area.
Messages	Text bubble states, timestamp, sender, unread marker, delivered/read indicators, typing state, empty/loading/error states.
New Conversation	Search modal, user selection, open-or-create chat behavior.
Groups	List, create modal, member selection, group info, admin actions.
Calls	Audio/video call screens, incoming call overlay, accept/reject, mute, camera toggle, end call, call status.
Shared Media	Image/video/file previews, gallery-like view, attachment metadata.
Starred	List/filter starred messages and open original conversation.
Profile	Profile information, avatar update, about text, edit/save behavior.
Settings	Theme, privacy, last seen, profile photo visibility, about, groups/privacy controls, blocked contacts, basic security/session controls, keyboard shortcuts.
Updates/Dashboard	Only implement data/interaction states visible in the supplied design and approved MVP behavior.
Responsive/Accessibility	Desktop-first fidelity plus agreed responsive breakpoints; keyboard focus, accessible labels, usable contrast and semantic controls.

5.3 Redux State Slices

Slice	Stores	Key actions/selectors
auth	user, token/session, status	loginStart/success/failure, logout, hydrate
chat	chat list, activeChatId, unread counts	setActiveChat, upsertChat, markRead, setLoading
messages	messages by chatId, pagination cursor, optimistic states	addMessage, updateMessage, removeMessage, markRead, setTyping
groups	group list/details/member state	createGroup, updateGroup, addMember, removeMember
calls	incoming/outgoing/active call state	startCall, acceptCall, rejectCall, endCall, setPeerState
notifications	in-app notifications/badges	pushNotification, markSeen
settings	theme and privacy toggles	setTheme, setPrivacy, setSecurity

6. Backend Requirements and Work Breakdown

Developer 2 owns the Express API, database models, business rules, authentication, file metadata, Socket.IO server behavior, call signaling, validation, logging and security middleware.

6.1 Backend Folder Structure

```

backend/src/
  app.ts
  server.ts
  config/
  routes/
    auth.routes.ts
    users.routes.ts
    chats.routes.ts
    messages.routes.ts
    groups.routes.ts
    calls.routes.ts
    uploads.routes.ts
    settings.routes.ts
  controllers/
  services/
  models/
    User.ts
    Chat.ts
    Message.ts
    Group.ts
    Call.ts
    Attachment.ts
    Block.ts
    Session.ts
  middleware/
    auth.ts
    validate.ts
    errorHandler.ts
    rateLimit.ts
  sockets/
    index.ts
    chat.socket.ts

```

call.socket.ts
 presence.socket.ts
 utils/
 types/

6.2 Backend Functional Areas

Area	Backend responsibilities
Auth	Register/login/logout, password hashing, token/session issue, auth middleware, input validation.
Users	Profile read/update, user search, online/last-seen state, blocked contacts.
Chats	Create/open 1:1 chats, list chats, membership checks, unread counters.
Messages	Persist messages, pagination, read/delivery state, star state, reactions if included, attachments.
Groups	Create/update group, add/remove members, member roles, group chat membership enforcement.
Real-time	Socket authentication, room membership, message events, typing/presence/read events.
Calls	Call signaling relay, session state, call logs.
Uploads	Generate/accept uploads, validate file type/size, persist metadata and storage URL.
Settings	Persist privacy, theme, about, blocked contacts and security/session preferences.
Security	CORS, input validation, authorization checks, rate limiting on auth endpoints, secure headers, structured errors, secret management.

7. API Contract and Data Contract

The API contract is the main coordination artifact. Both developers must treat docs/API_CONTRACT.md as the source of truth. An endpoint is not considered "done" until its request schema, response schema, authentication requirement and error behavior are documented.

7.1 REST Endpoints - MVP

Method	Endpoint	Auth	Purpose	Request	Response
POST	/api/auth/register	No	Create user	{name,email,password}	User + session/token
POST	/api/auth/login	No	Authenticate	{email,password}	User + token/session
POST	/api/auth/logout	Yes	End session	-	Success
GET	/api/users/me	Yes	Current user	-	User
GET	/api/users?search=	Yes	Search users	query	User[]
PATCH	/api/users/me	Yes	Update profile	partial user fields	User
GET	/api/chats	Yes	List chats	cursor?	Chat[] + cursor
POST	/api/chats	Yes	Create/open 1:1 chat	{userId}	Chat
GET	/api/chats/:chatId/ messages	Yes	Load messages	cursor?limit?	Message[] + cursor
POST	/api/chats/:chatId/ messages	Yes	Create message	message payload	Message
PATCH	/api/ messages/:messageId	Yes	Edit allowed message	{text}	Message
DELETE	/api/ messages/:messageId	Yes	Delete/soft-delete	-	Success
POST	/api/ messages/:messageId/ star	Yes	Toggle star	-	Message/Success
GET	/api/messages/starred	Yes	List starred	cursor?	Message[]
POST	/api/groups	Yes	Create group	group payload	Group
GET	/api/groups	Yes	List groups	-	Group[]

Method	Endpoint	Auth	Purpose	Request	Response
GET	/api/groups/:groupId	Yes	Group details	-	Group
POST	/api/groups/:groupId/members	Yes	Add members	{userIds[]}	Group
DELETE	/api/groups/:groupId/members/:userId	Yes	Remove member	-	Success
PATCH	/api/groups/:groupId	Yes	Update group	partial group fields	Group
GET	/api/calls	Yes	Call history	cursor?	Call[]
POST	/api/calls	Yes	Create call log	call metadata	Call
POST	/api/uploads/sign	Yes	Obtain upload target/signature	{type,size,name}	Upload target
GET	/api/settings	Yes	Load settings	-	Settings
PATCH	/api/settings	Yes	Update settings	partial settings	Settings

7.2 Standard API Response

```
{
  "success": true,
  "data": { ... },
  "message": "Optional human-readable message",
  "meta": { "cursor": "optional" }
}
```

7.3 Standard Error Response

```
{
  "success": false,
  "error": {
    "code": "AUTH_INVALID_CREDENTIALS",
    "message": "Invalid email or password",
    "fields": {}
  }
}
```

7.4 Contract Rules

- IDs are strings at the API boundary.
- Dates are ISO-8601 strings in UTC.
- Backend owns canonical timestamps and generated IDs.
- Frontend never invents server IDs for persisted entities.
- Every protected endpoint verifies authenticated user and resource access.
- Pagination uses cursor-based pagination for chat/message lists; the exact cursor format is an implementation detail documented by backend.
- All uploaded media has a type, size, storage URL/public identifier and owner/uploader ID in its metadata.

8. Database / Data Model Specification

MongoDB is the recommended persistence layer. Mongoose schemas should enforce required fields, enums, indexes and timestamps. The five-day MVP should keep schemas simple and avoid premature denormalization.

Collection / model	Core fields	Indexes / notes
User	_id, name, email, passwordHash, avatarUrl, about, status, lastSeenAt, createdAt, updatedAt	email unique; text/search index as appropriate
Chat	_id, type, participantIds, title?, avatarUrl?, lastMessageId?, createdAt, updatedAt	type + participant set; updatedAt
Message	_id, chatId, senderId, text?, type, attachments[], replyToId?, reactions[],	chatId+createdAt; senderId+createdAt

Collection / model	Core fields	Indexes / notes
	isStarred, deliveredTo[], readBy[], createdAt, updatedAt, deletedAt?	
Group	_id, name, description?, avatarUrl?, createdBy, createdAt, updatedAt	createdBy; updatedAt
GroupMember	_id, groupId, userId, role, joinedAt	groupId+userId unique
Call	_id, callerId, receiverId/groupId?, type, status, startedAt, answeredAt?, endedAt?, durationSec?	callerId+createdAt; receiverId+createdAt
Attachment	_id, ownerId, messageId, storageUrl, publicId?, mimeType, size, name, createdAt	messageId; ownerId+createdAt
Block	_id, ownerId, blockedUserId, createdAt	ownerId+blockedUserId unique
Session	_id, userId, refreshTokenHash/sessionId, userAgent?, ipHash?, expiresAt, revokedAt?	userId; TTL on expiry if used

8.1 Relationship Summary

User 1 --- * ChatMembership (or participantIds for MVP)
 Chat 1 --- * Message
 User 1 --- * Message (senderId)
 Group 1 --- * GroupMember * --- 1 User
 Message 1 --- * Attachment
 User 1 --- * Block (owner)
 User 1 --- * Call (caller / receiver)

9. Real-Time Messaging Specification

Socket.IO is used for low-latency events. REST remains the source for initial loading, pagination and durable state; Socket.IO handles live changes. This division avoids forcing the client to reconstruct the entire application state only from socket traffic.

9.1 Connection Lifecycle

1. Frontend creates a Socket.IO connection after valid authentication state is available.
2. Backend authenticates the socket using the agreed token/session mechanism.
3. On success, server attaches socket to a user-specific room such as user:{userId}.
4. When a chat is opened, frontend joins chat:{chatId} after the server verifies membership.
5. On disconnect, server updates transient presence state and the client may reconnect automatically.

9.2 Socket Events

Direction	Event	Payload essentials	Behavior
Client -> Server	chat:join	{chatId}	Verify membership then join room
Client -> Server	chat:leave	{chatId}	Leave room when appropriate
Client -> Server	message:send	{chatId,tempId?,type,text?,attach ments?}	Validate, persist, emit canonical message
Server -> Client	message:created	Message	Add/replace optimistic item using server ID
Client -> Server	message:read	{chatId,messageIds[]}	Persist read state and notify sender
Client -> Server	typing:start / stop	{chatId}	Ephemeral event; no DB write
Server -> Client	presence:update	{userId,status,lastSeenAt?}	Update online/offline state
Client -> Server	call:offer	{callId,toUserId,sdp,mediaType}	Signal WebRTC offer
Client -> Server	call:answer	{callId,toUserId,sdp}	Signal answer
Client -> Server	call:ice-candidate	{callId,toUserId,candidate}	Relay ICE candidate
Server -> Client	call:incoming	call invitation	Show incoming call UI

Direction	Event	Payload essentials	Behavior
Client -> Server	call:reject / call:end	{callId}	Update call state/log

9.3 Optimistic Messaging

- Frontend assigns a temporary ID (tempId) for immediate UI rendering.
- Backend returns the canonical persisted message with server ID and timestamps.
- Frontend replaces the optimistic message rather than creating a duplicate.
- On failure, frontend marks the optimistic message as failed and provides a retry action.

10. Calling and WebRTC Specification

The design contains Audio Call, Video Call and Call Logs. WebRTC should carry media; Socket.IO should be used only for signaling and call-state events. The backend should not proxy raw audio/video streams.

10.1 Call State Machine

```

idle -> dialing -> ringing -> connecting -> active -> ended
      |
      -> rejected / missed / failed

```

10.2 Frontend Responsibilities

- Request getUserMedia permissions and handle denied permissions cleanly.
- Create RTCPeerConnection with STUN/TURN configuration supplied by environment where needed.
- Exchange SDP offer/answer and ICE candidates through Socket.IO.
- Render local and remote tracks.
- Provide mute, camera toggle, speaker/output controls where supported, and end call.
- Update Redux call state so call UI is not scattered across unrelated components.

10.3 Backend Responsibilities

- Authenticate signaling socket connections.
- Validate caller/receiver relationship and call permissions.
- Relay signaling messages to the correct user/socket rooms.
- Persist call metadata and lifecycle timestamps.
- Prevent arbitrary users from sending signaling to unrelated users.

Five-day constraint

A functional call flow should be demonstrated on two devices/browsers by the end of Day 4. Network traversal issues can require TURN. Those infrastructure details should be treated as a known dependency rather than a reason to block all other modules.

11. Media / File Handling Specification

Media should not be stored as large binary blobs directly in MongoDB for this MVP. Use object storage such as Cloudinary and store metadata/URLs in MongoDB.

11.1 Supported MVP Types

Type	Examples	UI behavior
Image	JPG, PNG, WEBP	Thumbnail + lightbox/preview
Video	MP4/WebM subject to storage/browser support	Thumbnail + player
Document	PDF, DOCX, TXT and approved MIME types	File card + open/download
Avatar	JPG, PNG, WEBP	Circular/cropped avatar

11.2 Upload Flow

1. Frontend validates file type and approximate client-side size limit.
2. Frontend requests upload target/signature from backend.
3. Frontend uploads directly to storage if signed upload is used.
4. Frontend posts attachment metadata with the message or profile update.
5. Backend validates metadata again and persists the attachment URL/public identifier.
6. Conversation and Shared Media screens render the stored metadata.

12. Settings, Privacy and Security Requirements

12.1 Settings

Setting area	MVP behavior
Theme	Persist light/dark or selected theme and apply at app root.
Keyboard Shortcuts	Provide the visible shortcuts panel and wire only approved essential shortcuts in MVP.
Last Seen	Visibility toggle such as Everyone / Contacts / Nobody if matching product decisions.
Profile Photo	Visibility control plus profile photo update.
About	Update about text and visibility if required.
Groups	Group invite/member visibility controls as approved.
Blocked Contacts	List, block and unblock a user; blocked user cannot initiate the protected interactions defined by backend.
Security	Session list / logout-all capability may be implemented at MVP level.
Contact Info	Show profile details, shared media and block action.

12.2 Security Minimums

- Passwords hashed with a modern password hashing library (e.g., bcrypt/argon2).
- JWT/session secrets stored only in environment variables.
- CORS restricted to known frontend origins.
- Authorization checks on every resource containing user-specific data.
- Rate limiting on login/register/reset endpoints.
- Input validation for all REST and socket payloads.
- File type/size checks and safe upload handling.
- No secret values in frontend source or Git history.
- Production deployment uses HTTPS so API and WebSocket traffic are protected in transit.

13. Frontend-Backend Coordination and Integration Method

How to think about the workflow

The team does not build "all frontend first" and then "all backend". Instead, every feature has four states: Contract -> Build in parallel -> Integrate -> Verify.

13.1 Feature Delivery Loop

1. Pick one feature (e.g., Login).
2. Define request/response/socket contract.
3. Backend builds endpoint + model + validation.
4. Frontend builds UI + Redux + service using mock response if needed.

5. Backend publishes testable endpoint.
6. Frontend connects real endpoint.
7. Both test happy + failure paths.
8. Merge feature branches into develop.
9. Move to next feature.

13.2 Exact Coordination Example - Login

Step	Frontend developer	Backend developer
Contract	Implement LoginForm and Auth types from the contract.	Define POST /api/auth/login request/response and errors.
Parallel build	Use a mock login service returning typed user/session.	Implement User model, password check, auth service, endpoint.
Integration	Point service from mock to real API URL; update authSlice.	Expose local API and document base URL/CORS requirements.
Verification	Test success, invalid credentials, loading and logout.	Test endpoint with real user and error cases.
Handoff	Report API integration bugs with request/response sample.	Fix server errors; do not change response fields silently.

13.3 Shared Artifacts

Artifact	Owner	Update rule
docs/API_CONTRACT.md	Backend writes initial; both maintain	Any breaking change requires a PR note and frontend confirmation.
docs/SOCKET_CONTRACT.md	Backend + frontend	Payloads and event names are versioned/documented.
docs/DATA_MODEL.md	Backend	Frontend may reference only fields exposed by API.
README.md	Both	Setup commands, env names, ports, branch rules.
Integration checklist	Both	Updated at each daily checkpoint.

13.4 Definition of "API Ready"

- Endpoint implemented and reachable locally.
- Request/response examples documented.
- Authentication/authorization behavior confirmed.
- At least success + one failure case manually tested.
- Frontend can integrate without guessing field names.

14. Git / Branching / Pull Request Workflow

Repository rule

Do not work directly on main. Use develop as the shared integration branch, then merge a tested release candidate into main.

```

main
├─ develop
│   ├── feature/frontend-auth
│   ├── feature/frontend-chat
│   ├── feature/frontend-groups
│   ├── feature/frontend-settings
│   ├── feature/frontend-calls
│   └── feature/backend-auth

```

```

|- feature/backend-chat
|- feature/backend-groups
|- feature/backend-settings
└- feature/backend-calls

```

14.1 Workflow Commands

```

git switch develop
git pull origin develop
git switch -c feature/frontend-chat
# work, commit
git add .
git commit -m "feat(frontend): build chat workspace"
git push -u origin feature/frontend-chat
# open PR -> develop
# review + integration test -> merge

```

14.2 Commit Naming

- feat(frontend): ...
- feat(backend): ...
- fix(frontend): ...
- fix(backend): ...
- docs: ...
- refactor: ...

14.3 Merge Rules

- One logical feature per PR when practical.
- No API contract changes without notifying the other developer.
- No secret files committed.
- PR must pass local smoke test before merge.
- After merge, pull the latest develop before starting the next feature.
- Resolve conflicts with both developers present whenever the conflict touches a shared contract or shared config.

15. Five-Day Detailed Execution Plan

Critical sequencing

Day 1 starts with architecture + contracts for BOTH developers, then both build in parallel. Day 2 integrates auth. Day 3 integrates real-time chat + groups. Day 4 integrates media + calls. Day 5 is hard integration, settings, responsive QA, bug fixing and release preparation.

15.1 Day 1 - Foundation + UI Shell + Backend Skeleton

Time block	Frontend - Developer 1	Backend - Developer 2	Coordination checkpoint
Morning	Initialize Next.js, TS, Tailwind, Redux Toolkit; create route/layout shell; inspect and map all Figma screens to routes/components.	Initialize Node/Express/TS; connect MongoDB; create config, error handling, auth middleware skeleton; define models list.	30-45 min joint planning: freeze MVP scope, ports, env names, repository layout, API naming, ID/date rules.
Midday	Build app shell: sidebar, main workspace, chat list, active chat layout; use static/mock data.	Build User, Chat, Message schema drafts; create /health endpoint; create API router structure.	Share docs/API_CONTRACT.md and docs/DATA_MODEL.md. No implementation starts without base contracts.
Afternoon	Build Login/Register UI, profile skeleton, settings shell, reusable UI primitives.	Implement auth service, password hashing, register/login/logout endpoints; seed/test user helper if useful.	API smoke test together. Frontend confirms actual login request/response fields.

Time block	Frontend - Developer 1	Backend - Developer 2	Coordination checkpoint
Evening	Prepare Redux auth/chat/message slices; create API client and socket service stubs.	Finish auth validation/error responses; CORS; environment handling; initial Socket.IO server setup.	Checkpoint: frontend can point at backend; both commit/push to feature branches.
End of day deliverable	UI shell + auth pages + mock chat layout.	Working auth API + DB connection + socket server skeleton.	Merged develop branch contains buildable frontend and backend baseline.

15.2 Day 2 - Authentication + Chat UI + Core Chat APIs

Block	Frontend	Backend	Integration
Morning	Connect real register/login/logout APIs; protected routes; auth hydration.	Test auth end-to-end; finish authorization middleware and User API.	Login/register integrated before midday.
Midday	Build chat list, conversation header, message list, composer with mock messages.	Implement chat creation/list and message list/create APIs, pagination shape.	Frontend switches chat load from mock to GET /api/chats.
Afternoon	Redux chat/message state, optimistic message state, unread UI.	Implement message persistence, authorization, read state and star toggle.	Connect GET messages + POST message; verify no duplicate optimistic messages.
Evening	Error/empty/loading states; responsive adjustments.	Add initial Socket.IO rooms and message event handlers.	First real-time send/receive demo between two browser windows.

15.3 Day 3 - Real-Time + Groups + Profile/Contact

Block	Frontend	Backend	Integration
Morning	Typing indicator, presence, read/delivery UI; group list/create UI.	Socket auth, presence, typing, read events; group models/routes/services.	Live typing + presence + read state demo.
Midday	Group create modal, group details, member picker, group chat state.	Group create/update/member add/remove and authorization.	Create group and send a group message end-to-end.
Afternoon	Profile/contact info screens; block/unblock UI.	User profile endpoints; block collection and permission enforcement.	Update profile and block a user; verify blocked interaction rules.
Evening	Starred messages + shared media UI skeleton.	Attachment model/upload signature route + starred query.	End-to-end star/unstar and basic media metadata path.

15.4 Day 4 - Media + Calls + Dashboard/Updates

Block	Frontend	Backend	Integration
Morning	Implement image/video/document attachment UI and preview.	Implement storage upload flow, validation, attachment metadata persistence.	Upload real image/file, send in chat, open in Shared Media.
Midday	Audio/video call screens, incoming call UI, controls, call states.	Socket signaling: offer/answer/ICE; call lifecycle + log endpoints.	Two-browser audio call test.
Afternoon	Video call remote/local streams, call logs page.	Persist call records; handle reject/missed/ended states.	Two-browser video call test and call log verification.
Evening	Dashboard/Updates screens mapped from supplied design; basic filter/view/create/edit where approved.	Implement minimal update/status API only if required for final demo.	Freeze new feature work after checkpoint; move to QA backlog.

15.5 Day 5 - Settings + Full Integration + QA + Release

Block	Frontend	Backend	Integration
Morning	Theme, privacy, last-seen, profile-photo visibility, security/session UI, keyboard shortcuts.	Settings persistence, block/security/session endpoints, final validation/security checks.	Connect all settings and verify persistence after reload/login.
Late morning	Responsive fixes, skeletons, error	API consistency review, indexes, logs,	Run end-to-end regression from

Block	Frontend	Backend	Integration
	states, navigation, loading states.	rate limiting, error normalization.	clean browser state.
Afternoon	Fix UI defects from QA; accessibility/focus checks; polish against Figma.	Fix API/socket bugs, permissions, call state edge cases.	Joint defect triage. Only P0/P1 defects block release.
Evening	Production build and deployment config.	Production env, database, storage, CORS, WebSocket deployment checks.	Smoke test production; merge release candidate -> main.
Final deliverable	Integrated responsive frontend connected to live API/socket/WebRTC flow.	Stable API/database/realtime backend and deployable configuration.	Demo-ready MVP + README + environment example + known-limitations list.

16. Daily Integration Checkpoints

Day	Must prove working before moving on
Day 1	Both apps start locally; MongoDB connects; /health works; login API exists; frontend route shell builds; contracts documented.
Day 2	Register/login/logout works; authenticated chat list loads; messages load/send; socket receives at least one message.
Day 3	Typing/presence/read state works; group can be created; profile updates; block/unblock rule works; starred message works.
Day 4	Real file upload works; attachment renders; two-browser audio/video call works; call record appears; dashboard/update scope is frozen.
Day 5	Full smoke test passes from a clean account/session; settings persist; production build succeeds; critical bugs resolved; release branch/main updated.

16.1 15-Minute Daily Team Meeting

- Each developer reports: Done / Today / Blocked.
- Only discuss blockers that affect the other developer or the delivery sequence.
- Any contract change is written down immediately in docs.
- End with the exact feature selected for the next integration checkpoint.

17. Testing and Acceptance Criteria

17.1 Frontend Test Areas

Area	Checks
Navigation	All approved routes open; protected routes redirect when unauthenticated.
Forms	Validation, disabled/loading state, server errors, success feedback.
Chat UI	Scroll behavior, empty state, unread state, typing state, duplicate prevention.
Media	Supported image/video/file previews; failed upload UI.
Calls	Permission denial, ringing, accept/reject, mute/camera, end call, cleanup.
Settings	Changes persist after refresh/login; theme applies consistently.
Responsive	No major overflow/broken layout at agreed desktop/tablet/mobile breakpoints.
Accessibility	Buttons have labels; keyboard can reach primary controls; focus is visible; dialogs trap focus where required.

17.2 Backend Test Areas

Area	Checks
Auth	Valid/invalid credentials; duplicate email; unauthorized endpoint.
Authorization	User cannot read/write another user's private data or a chat they are not a member of.
Chat	Create/open idempotency; message persistence; pagination; read state.
Groups	Membership; admin permissions; invalid member changes.
Sockets	Authentication; room access; message broadcast; typing/presence cleanup.
Calls	Signal only reaches authorized recipient; call status transitions; call logs.
Uploads	File type/size validation; metadata integrity; unauthorized access rules.
Security	CORS, validation, rate limits, secure secrets, consistent error handling.

17.3 End-to-End Acceptance Scenarios

- User A registers and logs in, opens main workspace and sees initial chat list.
- User A searches User B, starts a 1:1 chat, sends a message, and User B sees it live.
- User B replies; User A sees the message without refresh; typing/read indicators update.
- User A creates a group with User B and another user; group conversation works.
- User A shares an image; recipient sees thumbnail; Shared Media can display it.
- User A stars a message; Starred Messages lists it and opens the original chat.
- User A starts audio/video call with User B; accept/reject/end states update and call log is stored.
- User changes theme/privacy/last-seen settings; values persist after reload.
- User blocks another user; blocked behavior matches the agreed backend rule.
- Clean production build starts and connects to production API/socket endpoints using deployment environment variables.

18. Definition of Done

18.1 Feature-level DoD

- UI matches agreed design state sufficiently for the MVP.
- Frontend types are defined; no avoidable any types for API contracts.
- Backend endpoint/socket event is documented.
- Loading, success, empty and failure states are handled.
- Authorization and validation are implemented.
- Feature works against a real backend, not only mock data.
- Feature has been tested by both developers in an integration scenario.
- No console-breaking errors and no secrets in source control.

18.2 Project-level DoD

- All Must-have Day 5 items in Section 2.2 are complete or explicitly documented as limited.
- Production environment variables are configured.
- README contains setup, env variable names, database setup, run commands and branch rules.
- .env.example is committed and real env files are ignored.
- Known limitations are documented.
- Release candidate has been smoke-tested after merging into main.

19. Risks, Constraints and Stretch Scope

Risk / constraint	Impact	Mitigation
Five-day schedule	High	Freeze MVP scope on Day 1; feature-by-feature integration; no late large features.
WebRTC network traversal	High	Use STUN/TURN configuration; test on two networks if possible; demo locally while production relay is configured.
API contract drift	High	Single API contract document; no silent response-field changes.
Git conflicts	Medium	Separate frontend/backend directories and feature branches; small PRs.
Large media files	Medium	Direct object storage upload; strict file size/type limits.
Figma details not fully specified	Medium	Implement visible design faithfully; record assumptions and unresolved questions in a decisions log.
Real-time race conditions	Medium	Use server-generated IDs/timestamps and idempotent client reconciliation.
Production deployment differences	Medium	Deploy a minimal end-to-end slice before the final day; keep separate environment variables for local/preview/production.

19.1 Scope Freeze Rule

After the Day 4 evening checkpoint, new features should not be added unless they are necessary to make an existing acceptance scenario work. New ideas go into the post-MVP list. This protects the five-day deadline.

20. Appendices - Checklists, Naming and Handoff Templates

20.1 Frontend Start-of-Day Checklist

- Pull latest develop.
- Check docs/API_CONTRACT.md for changes.
- Confirm backend base URL and current endpoints.
- Confirm today's target feature and integration time.
- Work only inside assigned branch.
- Write/finish UI against mock data immediately if backend is not ready.

20.2 Backend Start-of-Day Checklist

- Pull latest develop.
- Check frontend assumptions/bugs from previous day.
- Confirm endpoint/socket contract before changing payloads.
- Implement validation + authorization before happy-path polish.
- Provide curl/Postman examples or a simple test script.
- Notify frontend as soon as an endpoint is ready; do not wait until the end of the day.

20.3 Handoff Message Template

Feature: Chat Messages

Status: Ready for integration

REST: GET /api/chats/:chatId/messages

REST: POST /api/chats/:chatId/messages

Socket: message:send -> message:created

Auth: Bearer token / authenticated socket

Response example: documented in API_CONTRACT.md

Known limitation: pagination cursor is basic in MVP

Tested: Postman + two browser windows

20.4 Bug Reporting Template

Field	Example
Feature	Send message
Severity	P1
Steps	Open chat -> type -> Send
Expected	Message appears once and recipient receives live
Actual	Message appears twice
Evidence	Screenshot / console / request payload
Owner	Frontend / Backend
Status	Open / In progress / Ready for retest / Closed

20.5 Final Team Checklist - Before Main

- ☐ Frontend build succeeds.
- ☐ Backend starts and connects to production database/storage.
- ☐ Login works in production.
- ☐ 1:1 messaging works in two sessions.
- ☐ Group creation and group messaging work.
- ☐ Media upload/shared media works.
- ☐ Starred messages work.
- ☐ Audio/video call flow has been tested.
- ☐ Core settings persist.
- ☐ No .env secrets are committed.
- ☐ API and socket contracts are committed.
- ☐ Known limitations are documented.
- ☐ README is updated.
- ☐ Production smoke test completed after release merge.

Appendix A - Design Screen to Implementation Mapping

Design state	Suggested route	Frontend components	Backend integration
Profile	/profile	ProfilePage, ProfileForm, AvatarUploader	GET/PATCH /api/users/me
Initial Screen / Chat Workspace	/chat	ChatLayout, ChatList, ConversationPanel	GET /api/chats + socket events
Start New Conversation	/chat/new	NewConversationModal, UserSearch	GET /api/users?search= + POST /api/chats
Call Logs	/calls	CallHistory, CallItem	GET /api/calls
Group	/groups	GroupList, GroupDetails, GroupChat	GET /api/groups, group endpoints
Create Group	/groups/new or modal	CreateGroupModal	POST /api/groups
Audio Call	/calls/:id	AudioCallView, CallControls	Socket signaling + POST /api/calls
Video Call	/calls/:id	VideoCallView, CallControls	Socket signaling + POST /api/calls
Dashboard Updates	/updates	UpdatesList/View/CreateEdit	Approved update/status APIs
Dashboard Filter Chats	/chat?filter=...	ChatFilters	GET /api/chats filters
Contact Info	/contacts/:id	ContactInfoPanel	GET /api/users/:id equivalent read endpoint
Shared Content / Media	/chats/:id/media	SharedMediaGallery	Message/Attachment queries
Starred Messages	/starred	StarredMessages	GET /api/messages/starred
Settings	/settings	SettingsLayout	GET/PATCH /api/settings
Theme	/settings/theme	ThemeSettings	PATCH /api/settings
Privacy	/settings/privacy	PrivacySettings	PATCH /api/settings
Last Seen	/settings/privacy/last-seen	LastSeenControl	PATCH /api/settings
Profile Photo	/settings/privacy/photo	ProfilePhotoControl	PATCH /api/settings
About	/settings/about	AboutSettings	PATCH /api/settings
Groups Privacy	/settings/privacy/groups	GroupsPrivacyControl	PATCH /api/settings
Blocked Contacts	/settings/blocked	BlockedContacts	GET/POST/DELETE block endpoints
Security	/settings/security	SecuritySettings, Sessions	Session/security endpoints
Keyboard Shortcuts	/settings/shortcuts	ShortcutsPanel	Mostly frontend; persist approved preferences if needed
Icons	shared UI	IconButton/Icon registry	Frontend component library

Appendix B - Suggested Initial Task Board

ID	Owner	Task	Day	Status
F-01	FE	Project shell + routes	Day 1	Open
F-02	FE	Design tokens + reusable UI	Day 1	Open
F-03	FE	Auth UI + Redux auth	Day 1-2	Open
F-04	FE	Chat workspace UI	Day 1-2	Open
F-05	FE	Message UI + optimistic state	Day 2	Open
F-06	FE	Real-time events client	Day 2-3	Open
F-07	FE	Groups UI	Day 3	Open
F-08	FE	Profile/contact/block UI	Day 3	Open
F-09	FE	Media UI	Day 3-4	Open
F-10	FE	Calls UI/WebRTC client	Day 4	Open
F-11	FE	Settings UI	Day 5	Open
F-12	FE	Responsive + visual QA	Day 5	Open
B-01	BE	Server/Mongo config	Day 1	Open
B-02	BE	Auth APIs	Day 1-2	Open
B-03	BE	Users/profile APIs	Day 2-3	Open
B-04	BE	Chat/message APIs	Day 2	Open

ID	Owner	Task	Day	Status
B-05	BE	Socket message/presence/typing /read	Day 2-3	Open
B-06	BE	Groups APIs	Day 3	Open
B-07	BE	Uploads + attachments	Day 3-4	Open
B-08	BE	Call signaling + logs	Day 4	Open
B-09	BE	Settings/privacy/security	Day 5	Open
B-10	BE	Security + production config	Day 5	Open
I-01	BOTH	Daily integration demo	Every day	Open
I-02	BOTH	Production smoke test	Day 5	Open

Appendix C - What to Build First?

Answer in one line

Neither side builds the whole app first. Build the foundation and contract together on Day 1, then build frontend and backend feature-by-feature in parallel and integrate at the end of each feature/day.

Recommended order

1. Foundation: repo, environments, architecture, naming and contracts.
2. Authentication: frontend UI + backend API + first integration.
3. Core chat: chat UI + chat/message APIs + Socket.IO messaging.
4. Groups: group UI + group APIs + group sockets.
5. Media: attachment UI + storage/upload backend.
6. Calls: call UI + WebRTC client + Socket.IO signaling + call logs.
7. Settings/profile/privacy/security.
8. Final regression, production build and deployment.

Team execution principle

The frontend developer should never sit idle waiting for the backend, and the backend developer should never redesign the response shape after frontend integration. Use mock data on the frontend, documented contracts on both sides, and a fixed integration checkpoint every day.